

# Laboratório de Concorrência · Aula 10

André Tavares · RA 240664982 · Turma D

Gerado em 09/10/2026, 20:07:39 · tempo registrado no cronômetro: 32:48

## Preparação: Concorrência em cinco minutos

**0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.**

Na cooperação, uma tarefa depende da outra para continuar. Exemplo: numa cozinha, uma pessoa prepara os ingredientes e outra só começa a cozinhar quando eles ficam prontos. Na competição, duas pessoas precisam usar o mesmo recurso e têm que se organizar para não usá-lo ao mesmo tempo. Exemplo: duas pessoas tentando usar o único caixa eletrônico disponível.

## Prática 1: A condição de corrida

**Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?**

2.000.000

**1ª execução (n = 1.000.000)**

esperado: 2000000 obtido: 1401125

**2ª execução (n = 1.000.000)**

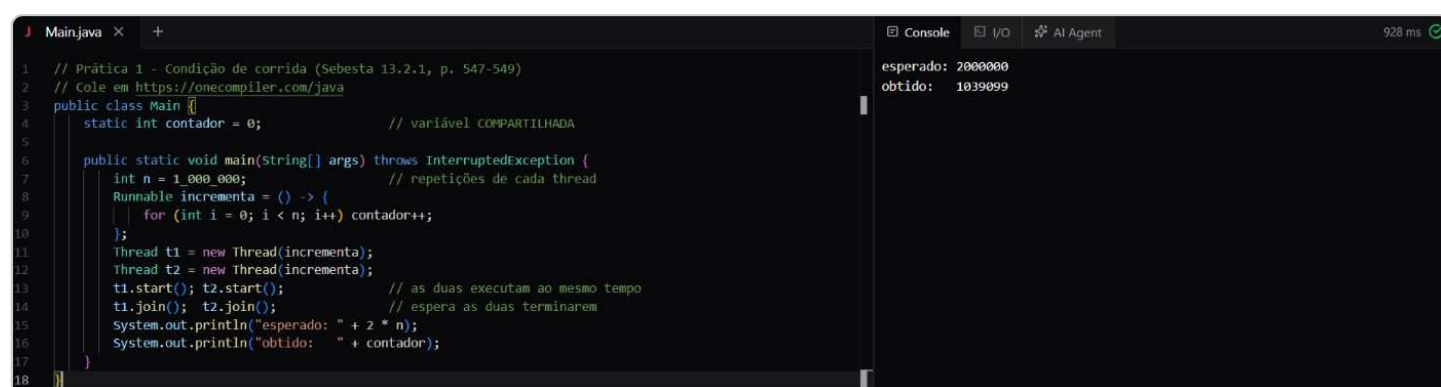
esperado: 2000000 obtido: 1163182

**3ª execução (n = 1.000.000)**

esperado: 2000000 obtido: 1039099

**Saída do programa**

*não respondido*



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

```
esperado: 2000000
obtido: 1039099
```

928 ms

**1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?**

Nem sempre o valor chega a 2.000.000 e as execuções podem apresentar resultados diferentes. Isso acontece porque contador++ envolve ler, somar e gravar, e as threads podem intercalar esses passos. Quando uma gravação sobrescreve a outra, uma atualização se perde. O resultado exato depende do escalonamento.

**1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.**

Por exemplo, o contador vale 5. A lê 5 e calcula 6, mas ainda não grava. B também lê 5 e calcula 6. A grava 6 e depois B grava 6. O contador aumentou só uma vez, embora as duas threads tenham incrementado; a gravação de B sobrescreveu o resultado de A.

### 1.3 Com $n = 1000$ , o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com  $n = 1000$ , o resultado pode até bater com o esperado em algumas execuções, porque a janela para ocorrer a disputa é menor, mas isso não garante que esteja correto. Em sistemas reais, o erro pode aparecer só sob carga e causar inconsistências, como saldo ou estoque incorreto. Passar nos testes não prova que não existe condição de corrida.

## Prática 2: Todas as respostas possíveis (Figura 13.1)

### Ordens testadas no simulador

AAABBB → 8

BBBAAA → 7

BBABAA → 4

BAAABB → 6

### Saída do programa

*não respondido*

```

5 from itertools import combinations
6
7
8 def executa(ordem):
9     total = 3
10    reg = {"A": 0, "B": 0}
11    passo = {"A": 0, "B": 0}
12    for t in ordem:
13        if passo[t] == 0:
14            reg[t] = total
15        elif passo[t] == 1:
16            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
17        else:
18            total = reg[t]
19        passo[t] += 1
20    return total
21
22
23 print("AAABBB ->", executa("AAABBB"))
24 print("BBBAAA ->", executa("BBBAAA"))
25 print("ABABAB ->", executa("ABABAB"))
26
27 resultados = {}
28 for posicoes in combinations(range(6), 3):
29     ordem = "".join("A" if i in posicoes else "B" for i in range(6))
30     resultados.setdefault(executa(ordem), []).append(ordem)
31
32 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
33 for valor in sorted(resultados):
34     ordens = resultados[valor]
35     print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
  
```

```

AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
  
```

### 2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis. No simulador/código aparecem 4 resultados diferentes: 4, 6, 7 e 8. Apenas 1 ordem dá 8: AAABBB, com A executando por completo antes de B.

### 2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Uma ordem que dá 4 é AABBBB. A lê 3 e calcula 4, mas ainda não grava. B lê 3, calcula 6 e grava 6. Por último, A grava o 4 que tinha calculado a partir do valor antigo. Assim, a gravação de A sobrescreve o resultado de B e o TOTAL termina em 4.

## Prática 3: Corrigindo a corrida

---

**Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?**

A versão sem sincronização provavelmente será a mais rápida, mas pode dar um resultado incorreto.

### Saída do programa

```
esperado:          2000000
sem sincronizacao: 1023568 em 2 ms
synchronized:      2000000 em 91 ms
AtomicInteger:     2000000 em 65 ms
```

```
esperado:          2000000
sem sincronizacao: 1007238 em 42 ms
synchronized:      2000000 em 49 ms
AtomicInteger:     2000000 em 55 ms
```

### 3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões com `synchronized` e `AtomicInteger` devem chegar ao valor correto. A versão sem sincronização pode perder atualizações. A versão correta costuma ser mais lenta porque precisa coordenar o acesso ao contador: `synchronized` faz as threads esperarem pela seção crítica, e `AtomicInteger` usa uma operação atômica com custo de coordenação. Os tempos variam conforme a máquina e a execução.

### 3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um `AtomicInteger` em cada saldo resolve? Por quê? O que seria preciso?

Não necessariamente. Cada `AtomicInteger` protege a atualização de um saldo isoladamente, mas a transferência precisa debitar uma conta e creditar a outra como uma única operação lógica. É preciso sincronizar as duas contas na mesma transação, com uma ordem consistente de bloqueio para evitar corrida e deadlock, e garantir que a transferência seja concluída por inteiro ou desfeita.

## Prática 4: Impasse (deadlock)

---

**Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?**

Provavelmente não. A pode ficar segurando X enquanto espera Y, e B pode ficar segurando Y enquanto espera X. Assim, nenhuma consegue chegar à mensagem de término.

### Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas
```

```
A terminou
B terminou
sem impasse
```

### 4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A adquire o recurso X e fica esperando Y. Ao mesmo tempo, a thread B adquire Y e fica esperando X. Como cada uma segura o recurso de que a outra precisa, nenhuma consegue avançar nem liberar o que já possui: isso é um deadlock.

## 4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se todas as threads adquirirem os recursos na mesma ordem, por exemplo X antes de Y, não ocorre o ciclo de espera mostrado no exemplo. Uma thread pode esperar X, mas não ficará segurando Y enquanto espera X. Isso pode acontecer quando dois serviços ou processos precisam bloquear os mesmos registros de banco de dados em ordens diferentes.

## Prática 5: Mensagens em vez de memória compartilhada

**Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?**

Com 3 trabalhadores, espero cerca de 200 ms, pois cada trabalhador pode executar duas tarefas de 100 ms. Com 1 trabalhador, cerca de 600 ms, com 6 trabalhadores, cerca de 100 ms. Na prática, os tempos podem ser um pouco maiores por causa do escalonamento e da sobrecarga.

### Tempo com 1 trabalhador

600ms

### Tempo com 3 trabalhadores

200ms

### Tempo com 6 trabalhadores

100ms

### Saída do programa

```
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 2 fez a tarefa 4
trabalhador 3 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

## 5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, as 6 tarefas são feitas em sequência, então o tempo fica perto de 600 ms. Com 3, elas são divididas e o tempo esperado fica perto de 200 ms. Com 6, cada trabalhador pode fazer uma tarefa e o tempo fica perto de 100 ms. Não é necessário synchronized porque as tarefas são distribuídas por um canal e cada trabalhador recebe uma tarefa; não há várias goroutines alterando o mesmo contador compartilhado.

### Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2
```

1

## 5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

No canal sem buffer, o envio `ch <- 1` espera até que outra goroutine receba a mensagem. Como a mesma goroutine tenta enviar antes de executar a leitura, ela fica bloqueada e ocorre deadlock. Com buffer de tamanho 1, a mensagem pode ficar guardada no canal, o envio termina e a leitura seguinte consegue recebê-la.

## Prática 6: O compilador como guarda

**Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?**

Acredito que o Rust não vai permitir compilar o programa, porque a thread tenta acessar e alterar uma variável que pertence a main sem uma forma segura de compartilhamento.

### Saídas (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
  → HelloWorld.rs:7:27
```

```
|
7 |     let t = thread::spawn(|| {
|                               ^^ may outlive borrowed value `contador`
8 |         contador += 1;           // a thread altera a variável de main
|         ----- `contador` is borrowed here
|
```

```
note: function requires argument type to outlive `'static`
```

```
  → HelloWorld.rs:7:13
|
7 |     let t = thread::spawn(|| {
|     ^^^^^^^^^^^^^^^^^^^^^^^^
8 | |         contador += 1;           // a thread altera a variável de main
9 | |     });
| |     ^
help: to force the closure to take ownership of `contador` (and any other referenced
variables), use the `move` keyword
|
7 |     let t = thread::spawn(move || {
|                               +++++
```

```
help: to force the closure to take ownership of `contador` (and any other referenced
variables), use the `move` keyword
```

```
7 |     let t = thread::spawn(move || {
|                               +++++
```

```
error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
  → HelloWorld.rs:11:20
```

```
|
7 |     let t = thread::spawn(|| {
|         -               -- mutable borrow occurs here
|         |
|         |
8 | |         contador += 1;           // a thread altera a variável de main
| |         ----- first borrow occurs due to use of `contador` in closure
9 | |     });
| |     - argument requires that `contador` is borrowed for `'static`
10 |         t.join().unwrap();
11 |         println!("{}", contador);
|         ^^^^^^^^^ immutable borrow occurs here
|
```

```
note: requirement that the value outlives `'static` introduced here
```

```
  →
/rustc/48a229ceaefd4985c50990b14116b6d856af0985/library/std/src/thread/functions.rs:128:14
```

```
error: aborting due to 2 previous errors
```

Some errors have detailed explanations: E0373, E0502.

For more information about an error, try `rustc --explain E0373`.

```
warning: value captured by `contador` is never read
```

```
→ HelloWorld.rs:8:9
```

```

|
8 |         contador += 1;           // a thread altera a variável de main
|         ^^^^^^^^^
|
= help: did you mean to capture by reference instead?
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default

```

```
warning: value assigned to `contador` is never read
```

```
→ HelloWorld.rs:8:9
```

```

|
8 |         contador += 1;           // a thread altera a variável de main
|         ^^^^^^^^^^^^^^^^^
|
= help: maybe it is overwritten before being read?

```

```
warning: 2 warnings emitted
```

## Saída do programa

```
contador = 400000
```

### 6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread recebe a propriedade de uma cópia do número contador. Ela incrementa essa cópia, enquanto a variável original de main continua valendo 0. Por isso o programa compila, mas imprime 0.

### 6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o problema da condição de corrida aparece durante a execução e pode passar despercebido nos testes. Neste exemplo em Rust, o compilador identifica antes que o acesso à variável não é seguro. Isso ajuda a evitar erros, mas exige que o programador entenda e siga as regras de propriedade e compartilhamento da linguagem.

## Prática 7: Uma thread só: o laço de eventos

### Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Acho que a ordem será A, D, C e B: A e D são impressas diretamente, a promessa executa antes do timer e o callback de B fica para depois.

## Saída do programa

```

A
D
C
B
E: timer de 10 ms rodou depois de 300 ms

```

**7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?**

A, D, C e B. A e D são impressas durante a execução síncrona do código. Depois, a tarefa C roda na fila de microtarefas, que tem prioridade sobre os callbacks de setTimeout. Por isso B, mesmo com atraso 0, sai depois de C.

**7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?**

O timer não interrompe o código que está rodando. A função ocupado bloqueia a única thread por cerca de 300 ms, então o callback do timer só consegue executar quando esse cálculo termina, mesmo que o atraso configurado seja de 10 ms. Em um servidor Node.js, um cálculo pesado pode atrasar as respostas de muitos usuários; por isso, tarefas pesadas devem ser divididas ou executadas fora do laço de eventos, por exemplo em workers.

**Fechamento: Quem impede a corrida?**

---

**Java sem sincronização**

Ninguém: há corrida

**Java com synchronized**

O programador, com sincronização

**Go com canais**

O programador, com sincronização

**Rust com Arc e Mutex**

O programador, com sincronização

**JavaScript**

Só existe uma thread

**8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?**

Pode ocorrer uma condição de corrida: as duas compras leem que ainda existe um lugar e ambas confirmam, vendendo duas vezes o mesmo ingresso. Eu evitaria isso fazendo a reserva e a atualização do estoque em uma transação atômica no servidor/banco de dados, com bloqueio ou uma atualização condicional que só permita a compra se ainda houver lugar.

**8.2 Em uma frase: o que mais surpreendeu você nesta atividade?**

O que mais me surpreendeu foi perceber que um código aparentemente simples pode produzir resultados diferentes dependendo da ordem em que as tarefas executam.